Data Structure

Lecture 1

Outline

- Introduction to Data Structure
- Types of data structure
  - primitive and non primitive
  - linear and non linear DS
- Data structure operations
- Linear Arrays
  - Definition and concepts
  - Representation
  - Operations on arrays: traversing, inserting, operations.
- Stacks
  - Definition and concepts,
  - Representations
  - Operations on stack: Push and Pop.

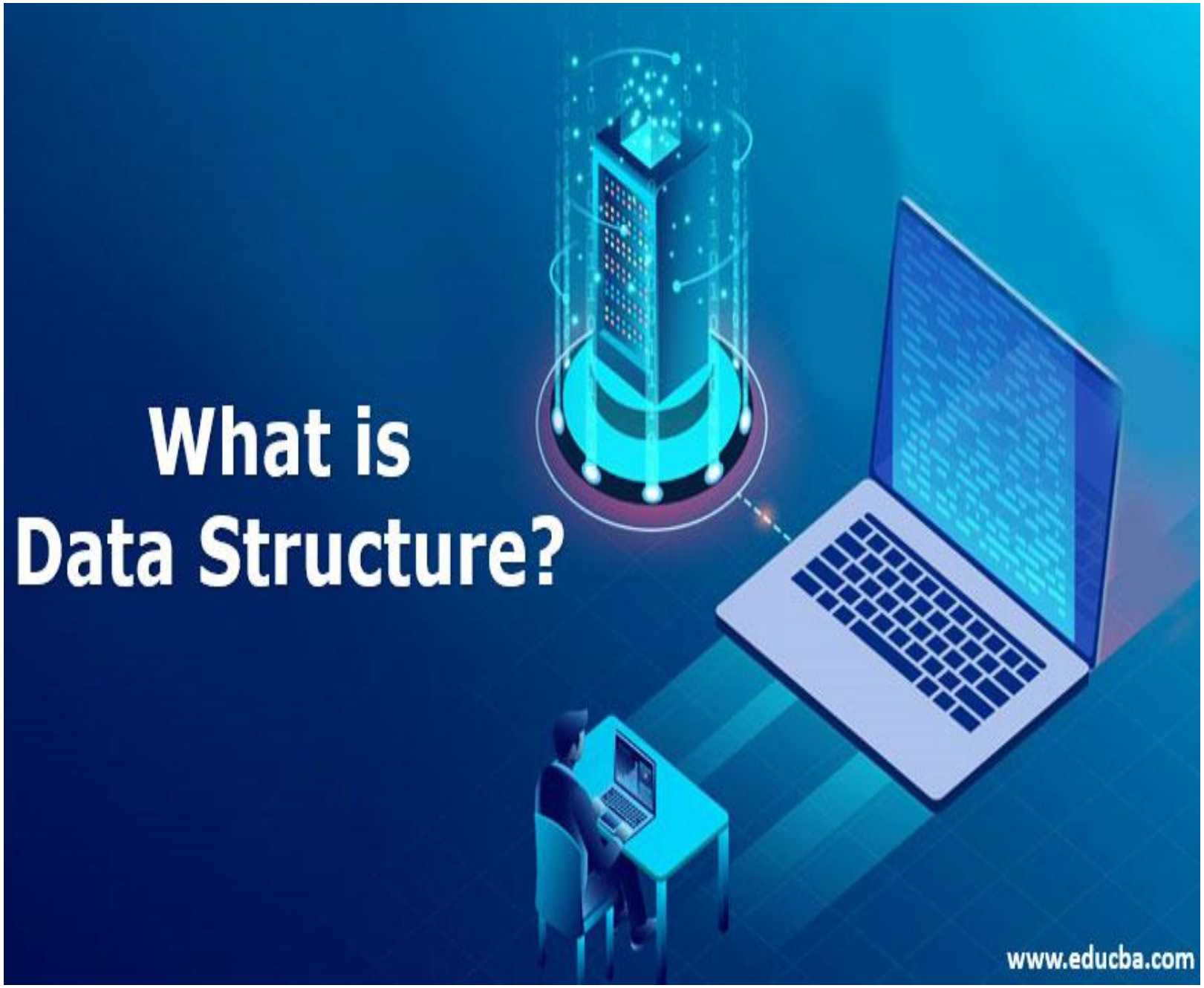
Introduction to Data Structure

- Computer is an electronic machine which is used for data processing and manipulation.
- When programmer collects such type of data for processing, he would require to store all of them in computers main memory.
- In order to make how computer work we need to know

  Representation of data in computer.

  Accessing of data.

  How to solve problem step by step.
- For doing all of this task we used Data Structure



What is Data Structure

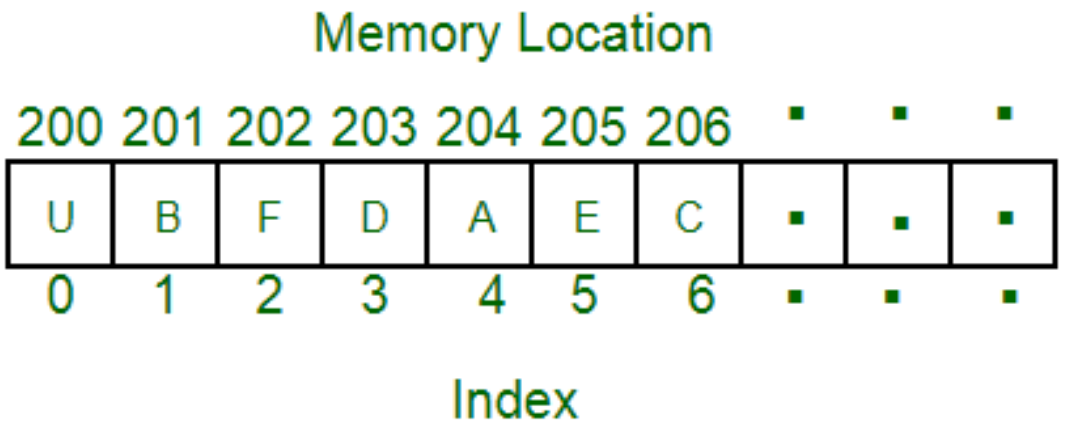
- A data structure is a specialized format for organizing, processing, retrieving and storing data.
- In computer programming, a data structure may be selected or designed to store data for the purpose of working on it with various algorithms.

What is Data Structure

- Data Structure can be defined as the group of data elements which provides an efficient way of storing and organizing data in the computer so that it can be used efficiently.
- examples of Data Structures are arrays, Linked List, Stack, Queue, etc.
- Data Structures are widely used in almost every aspect of Computer Science i.e. Operating System, Compiler Design, Artificial intelligence, Graphics and many more.
- Data Structures are the main part of many computer science algorithms as they enable the programmers to handle the data in an efficient way.
- It plays a vital role in enhancing the performance of a software or a program as the main function of the software is to store and retrieve the user's data as fast as possible

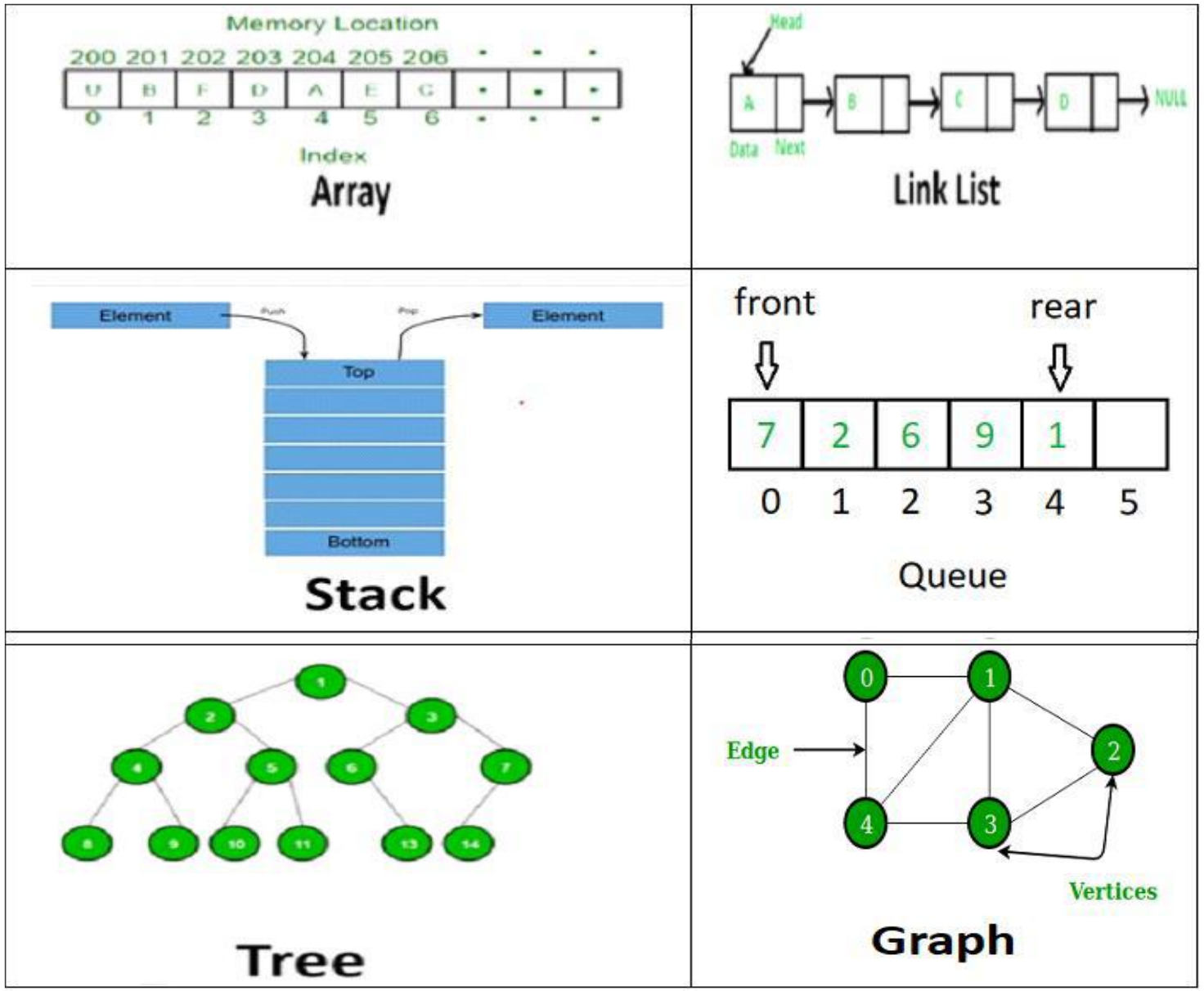
Data Structure

- A **data structure** is a particular way of organizing data in a computer so that it can be used effectively.
- For example, we can store a list of items having the same data-type using the *array* data structure.



Data Structure…

- **General Data Structures:**



- The representation of particular data structure in the main memory of a computer is called as storage structure.
- The storage structure representation in auxiliary memory is called as file structure.
- It is defined as the way of storing and manipulating data in organized form so that it can be used efficiently
- Data Structure mainly specifies the following four characteristics:

  1.Organization of Data: It describes the arrangement of data structure elements in memory.

  2.Accessing Methods: It describes the data structure's access methods for all the elements.

  3.Degree of Association: It deals with the connection or association of a data element with other elements.

  4.Processing Methods: It describes the execution of operations like insertion, deletion, updation etc. in the data structure.

- **Algorithm + Data Structure = Program**
- **Data Structure study Covers the following points**

  1) Amount of memory require to store

  2) Amount of time require to process

  3) Representation of data in memory

  4) Operations performed on data

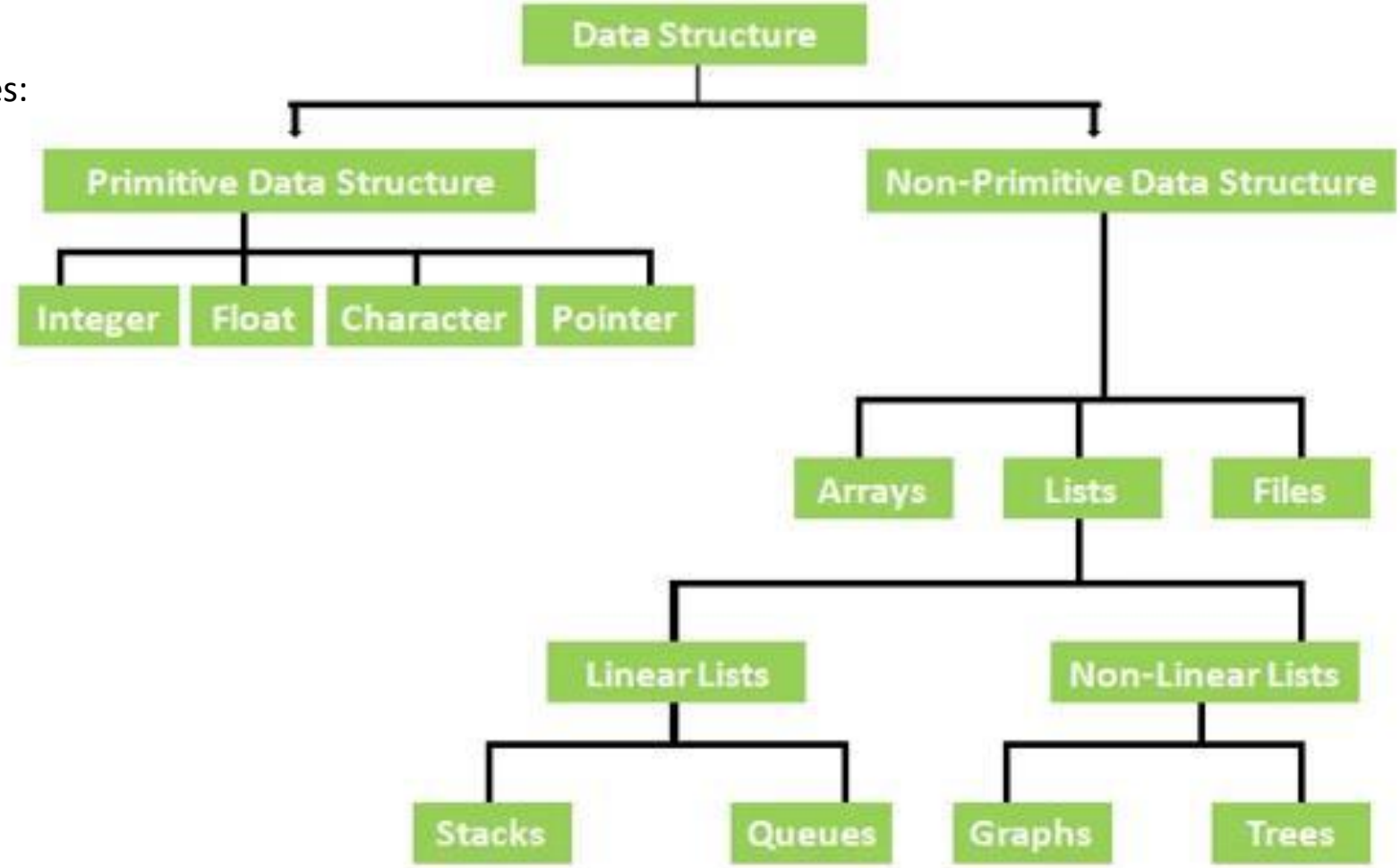
Types of Data Structures

The DS are divided into two types:

1) Primitive
2) Non primitive

Non primitive divided into two type

1) Linear DS
2) Non linear DS



❑ Primitive Data Type

- Primitive Data Structure are basic structure and directly operated upon by machine instructions.
- Primitive Data Structures of any programming language are the data types that are predefined in that programming language. The type and size of those data types are already defined.
- The main primitive data structures which are defined in mostly all programming languages are Integer, Float, Character, Pointers, and Boolean etc.
- The value to the primitive data structure is provided by the programmer.
- Primitive data structures have different representations on different computers.
- Integers, floats, character and pointers are example of primitive data structures.
- These data types are available in most programming languages as built in type.
- **Integer:** The integer data type contains the numeric values. It contains the whole numbers that can be either negative or positive. When the range of integer data type is not large enough then in that case, we can use long.
- **Float:** The float is a data type that can hold decimal values. When the precision of decimal value increases then the Double data type is used.
- **Character:** It is a data type that can hold a single character value both uppercase and lowercase such as 'A' or 'a'.
- **Pointer:** A variable that hold memory address of another variable are called pointer.
- **Boolean:** It is a data type that can hold either a True or a False value. It is mainly used for checking the condition.

Non Primitive Data Type

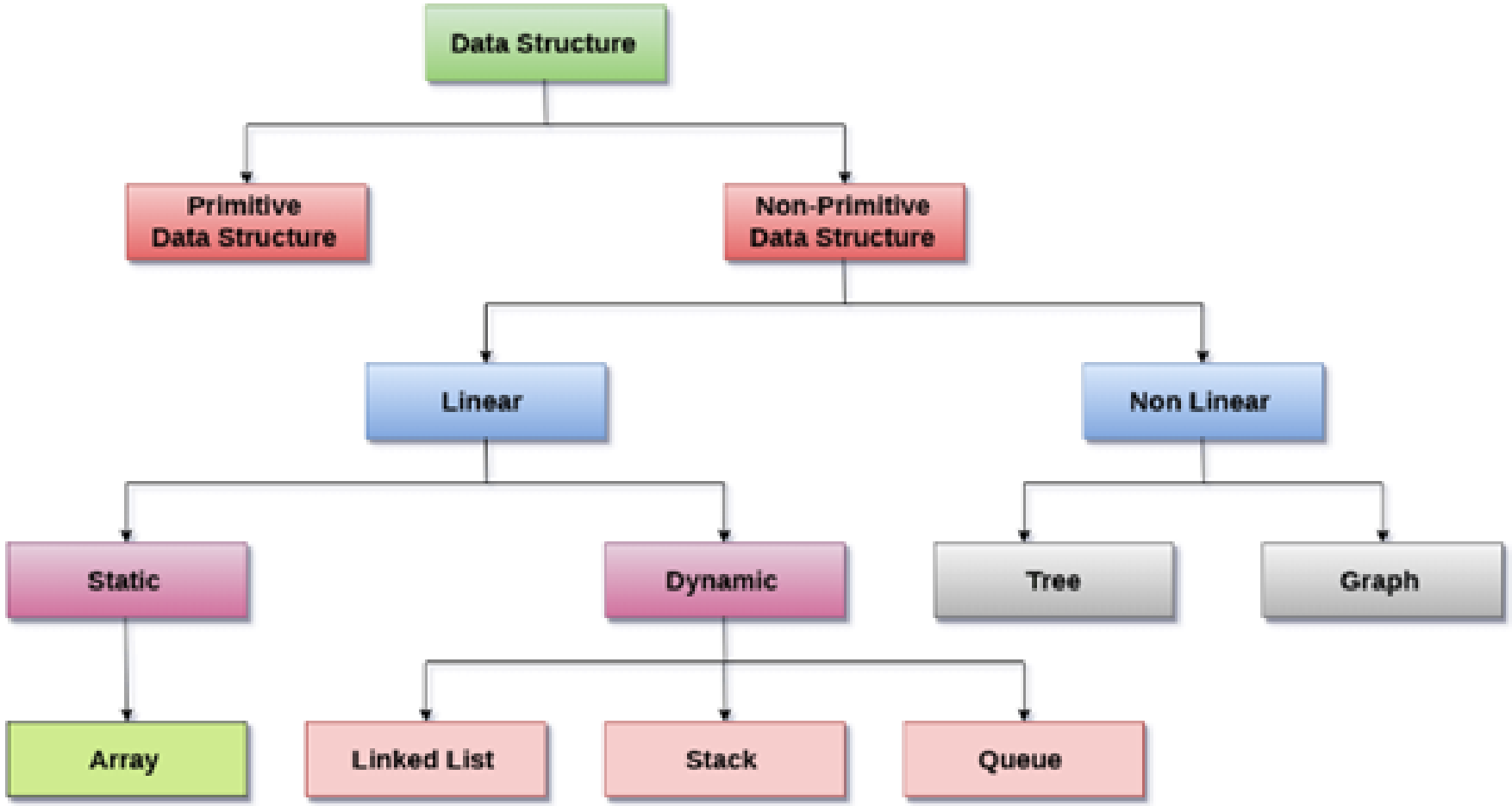
- These are more sophisticated data structures.
- These are derived from primitive data structure.
- The non – primitive data structures emphasize structuring of a group of homogeneous or heterogeneous data items.
- Example of non – primitive data types are Array, List, and File etc.
- A non – primitive data type is further divided into Linear and non – Linear data structure.

Array: An array is a fixed size sequenced collection of elements of the same data type.

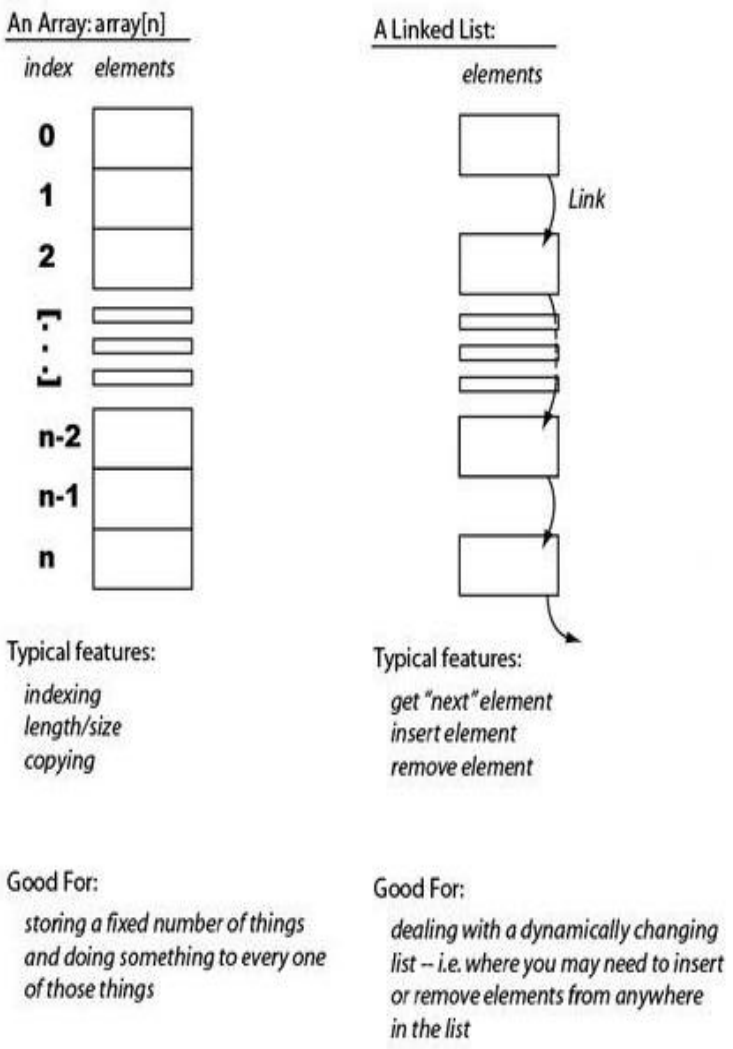
List: An ordered set containing variable number of elements is called as List.

File: A file is a collection of logically related information. It can be viewed as a large list of records consisting of various fields.

Non Primitive Data Type



Linear Data Structures



- A linear data structure simply means that it is a storage format of the data in the memory in which the data are arranged in contiguous blocks of memory.
- Example is the array of characters it represented by one character after another.
- In the linear data structure, member elements form a sequence in the storage.
- There are two ways to represent a linear data structure in memory.

  static memory allocation

  dynamic memory allocation

The possible operations on the linear data structure are:

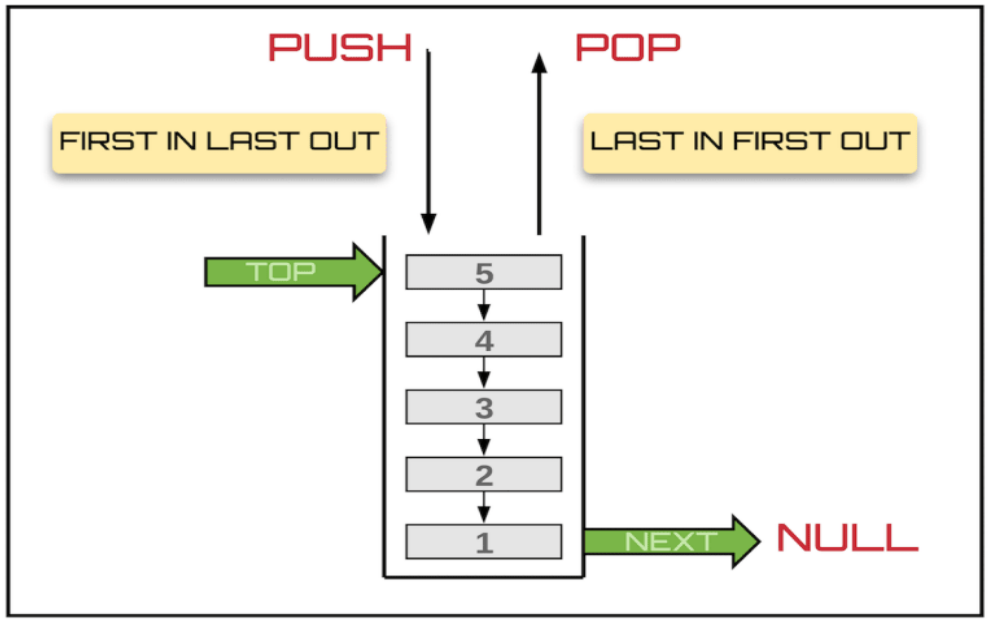
1) Traversing 2) Insertion 3) Deletion 4) searching 5) sorting

6) merging

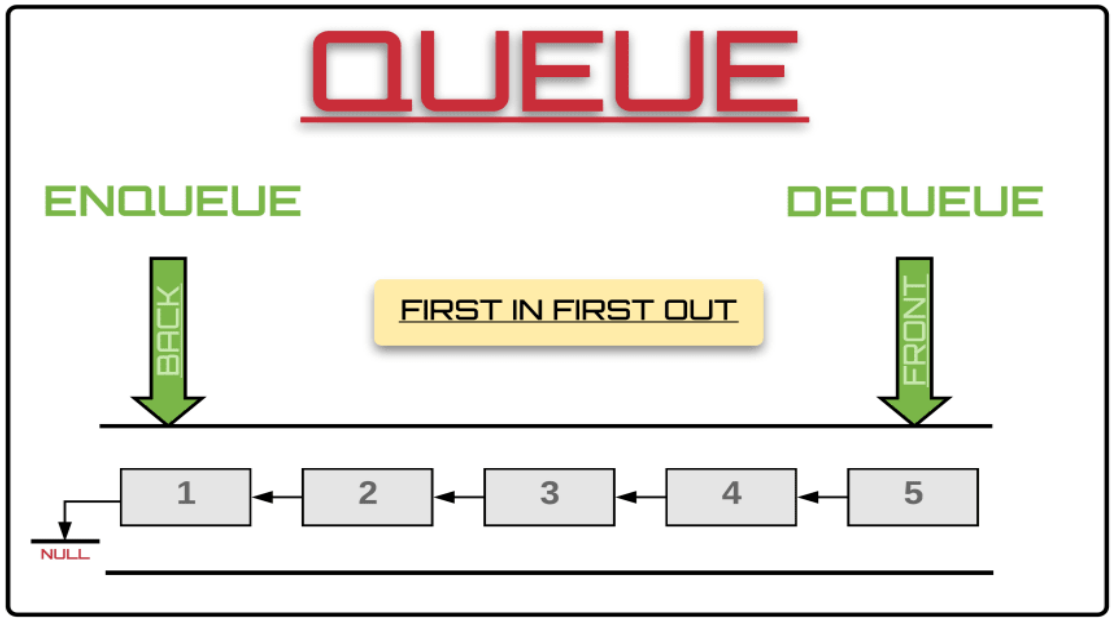
Types of Linear Data Structures:

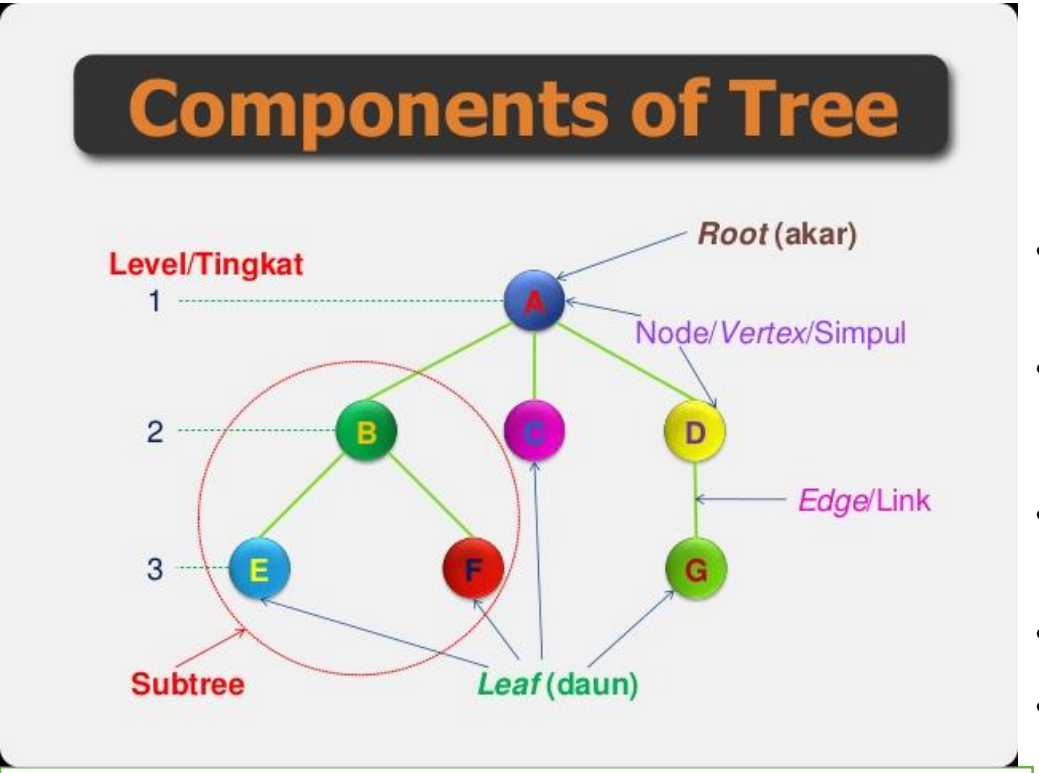
- **Array:** An Array is a ***collection of similar type of data items*** and each data item is called an element of the array. The data type of the element may be any valid data type like char, int, float or double.
- The elements of array share the same variable name but each one carries a different index number known as subscript.
- The array can be one dimensional, two dimensional or multidimensional.
- **Linked List:** Linked List is a linear data structure which is used to maintain a list in the memory. It can be seen as the collection of nodes stored at non-contiguous memory locations. Each node of the list contains a pointer to its adjacent node.
- **Stack:** Stack is a linear list in which insertion and deletions are allowed only at one end, called **top**.
- It follows **Last-In-First-Out (LIFO)** principle for storing the data items.
- **Queue:** Queue is a linear list in which elements can be inserted only at one end called **rear end** and deleted only at the other end called **front end.**
- Queue is opened at both ends therefore it follows **First-In-First-Out (FIFO)** principle for storing the data items.

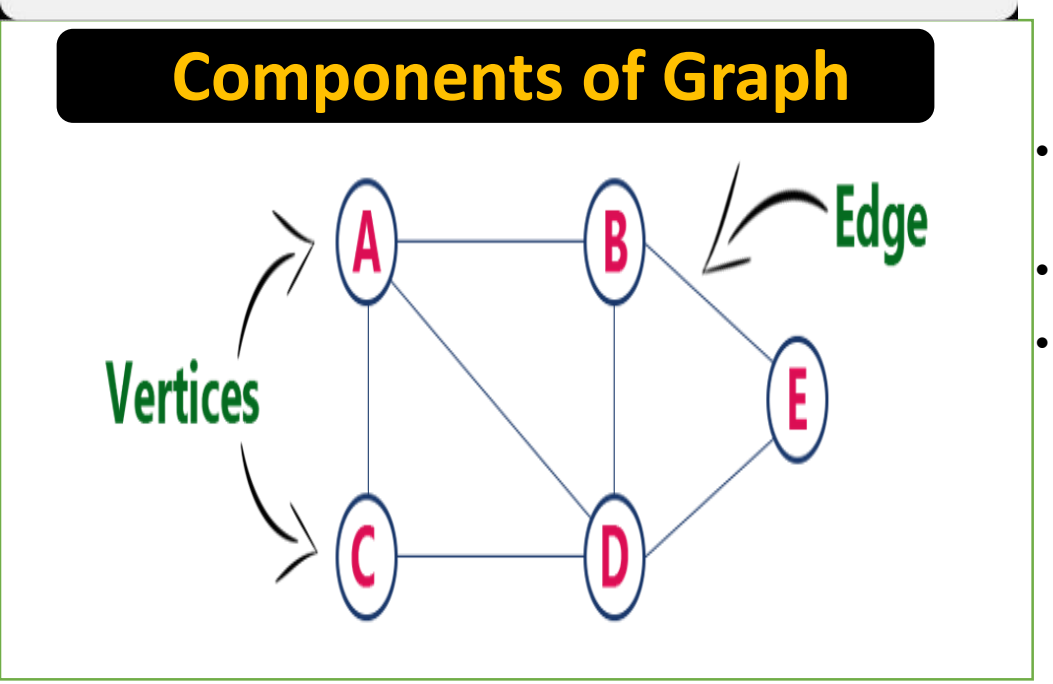
STACK


PUSH
POP
FIRST IN LAST OUT
LAST IN FIRST OUT
TOP
5
4
3
2
1
NEXT
NULL

QUEUE


ENQUEUE
DEQUEUE
BACK
FIRST IN FIRST OUT
FRONT
1
2
3
4
5
NULL





Non-Linear Data Structure

- Non linear DS are those data structure in which data items are not arranged in a sequence.
- Example on Non Linear DS are Tree and Graph.

TREE

- A Tree can be define as finite data items (nodes) in which data items are arranged in branches and sub branches
- Tree represent the hierarchical relationship between various elements
- Tree consist of nodes connected by edge, the represented by circle and edge leaves connecting to circle.

Graph

- Graph is collection of nodes (information) and connecting edges (Logical relation) between nodes.
- A tree can be viewed as restricted graph
- Graph have many types: 1) Simple graph 2) Mixed graph 3) Multi graph 4) Directed graph 5) Un-directed graph

Difference Between Linear and Non Linear Data Structure

Linear Data Structure

- Every item is related to its previous and next item.
- Data is arranged in linear sequence.
- Data items can be traversed in a single run
- E.g. Array, Stacks, Linked list, Queue
- Implementation is easy.

Non – Linear Data Structure

- Every item is attached with many other items.
- Data is not arranged in sequence.
- Data cannot be traversed in a single run.
- E.g. Tree, Graph
- Implementation is difficult.

Data Structure Major Operations

(1) Traversing: Every data structure contains the set of data elements. Traversing the data structure means visiting each element of the data structure in order to perform some specific operation like searching or sorting.

(2) Insertion: Insertion can be defined as the process of adding the elements to the data structure at any location.

If we try to add an element to an already filled data structure then overflow condition occurs.

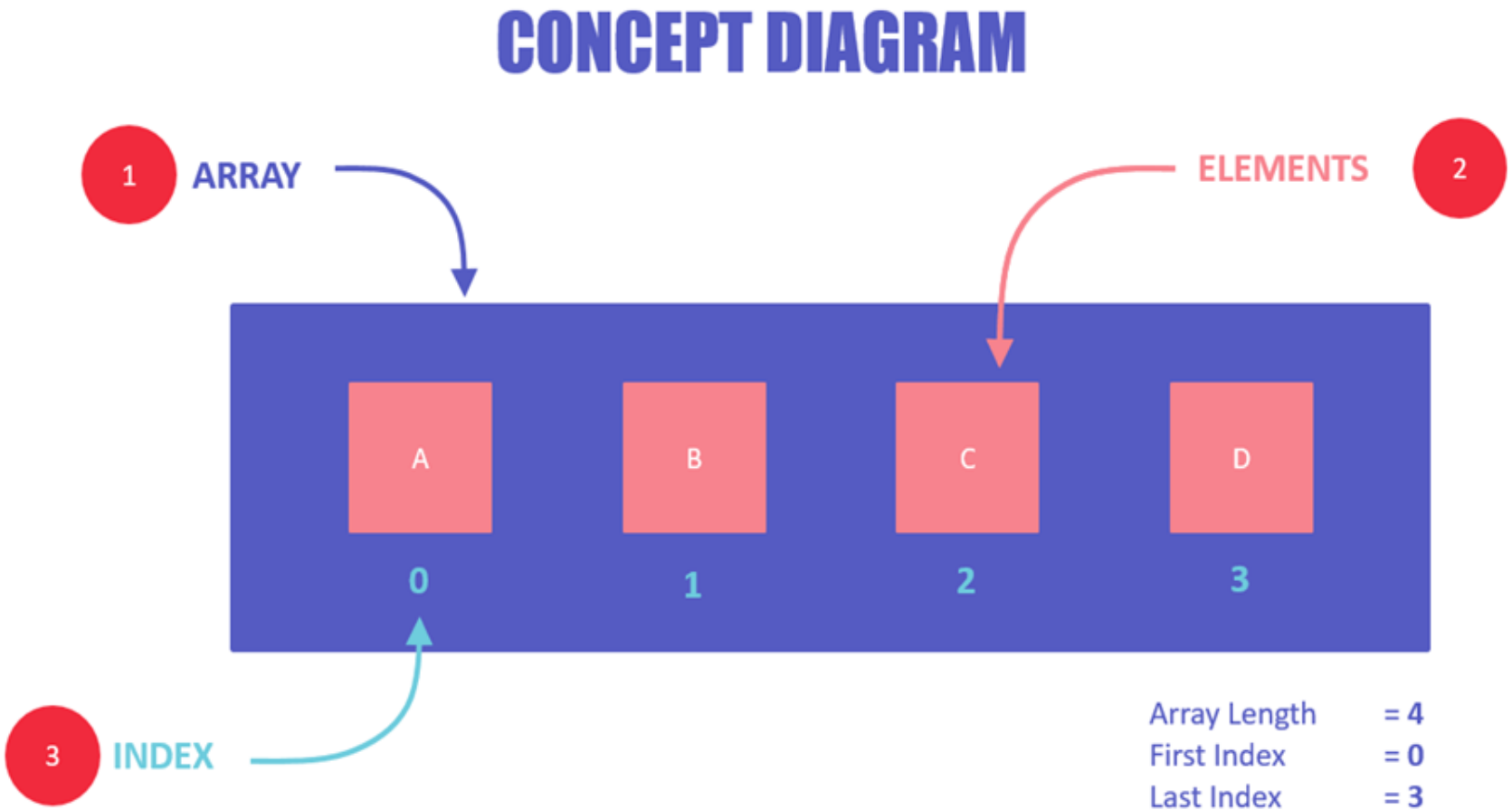
(3) Deletion: The process of removing an element from the data structure is called Deletion. We can delete an element from the data structure at any random location.

If we try to delete an element from an empty data structure then underflow condition occurs.

(4) Searching: The process of finding the location of an element within the data structure is called Searching. There are various algorithms to perform searching e.g: Linear Search and Binary Search.

(5) Sorting: The process of arranging the data structure in a specific order is known as Sorting. There are many algorithms that can be used to perform sorting, for example, insertion sort, selection sort, bubble sort, etc.

(6) Merging: When two lists List A and List B of size M and N respectively, of similar type of elements, clubbed or joined to produce the third list, List C of size (M+N), then this process is called merging.



What are Arrays?

Array is a container which can hold a fix number of items and these items should be of the same type.

Most of the data structures make use of arrays to implement their algorithms.

•Following are the important terms to understand the concept of Array.

Element − Each item stored in an array is called an element.

Index − Each location of an element in an array has a numerical index, which is used to identify the element.

1. An array is a container of elements.
2. Elements have a specific value and data type, like "ABC", TRUE or FALSE, etc.
3. Each element also has its own index, which is used to access the element.

UNDERSTAND SYNTAX

PYTHON

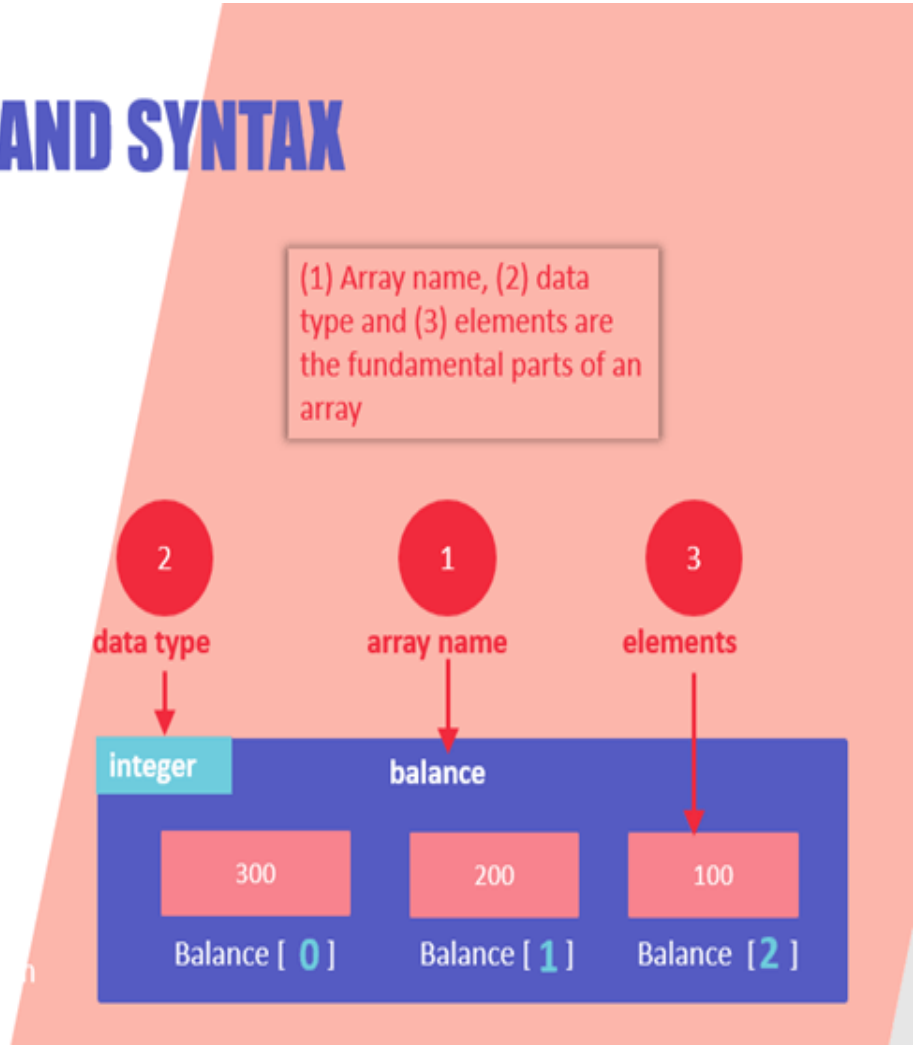
```
balance = array.array( 'i', [300,200,100] )
```

array name — data type — elements

C++

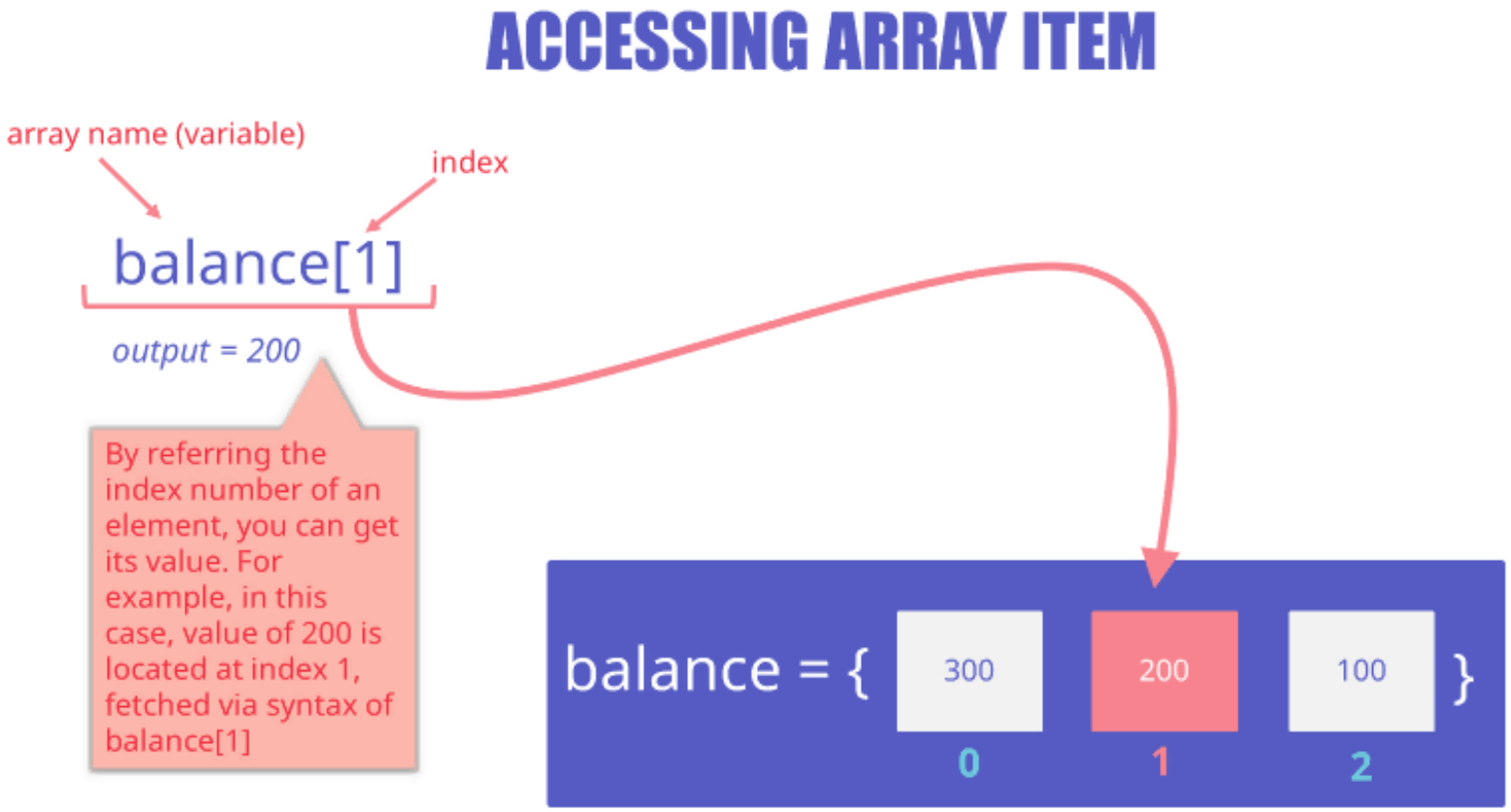
```
int balance[3] = { 300, 200, 100 }
```

data type — array name — elements



- **Elements are stored at contiguous memory locations.**
- **An index is always less than the total number of array items.**
- **In terms of syntax, any variable that is declared as an array can store multiple values.**
- **Almost all languages have the same comprehension of arrays but have different ways of declaring and initializing them.**
- **However, three parts will always remain common in all the initializations, i.e., array name, elements, and the data type of elements.**

- **Array name:** necessary for easy reference to the collection of elements
- **Data Type:** necessary for type checking and data integrity
- **Elements:** these are the data values present in an array



How to access a specific array value?

You can access any array item by using its index

Syntax

arrayName[indexNum]

Example

balance[1]

Here, we have accessed the second value of the array using its index, which is 1. The output of this will be 200, which is basically the second value of the balance array.

- **Array Representation**

- Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.